

Simulation with Vectors

In an earlier chapter, you wrote a Python program that simulated the flight of a hammer to predict its altitude. Your simulation dealt only with scalars. Now, you are ready to create simulations of positions, velocities, accelerations, and forces as vectors.

In this chapter, you are going to simulate two moons that, as they wandered through the vast universe, get caught in each other's gravity well. We will assume there are no other forces acting upon the moons.

1.1 Force, Acceleration, Velocity, and Position

We talked about the magnitude of a gravitational attraction between two masses:

$$F = G \frac{m_1 m_2}{r^2}$$

where F is the magnitude of the force in newtons, m_1 and m_2 are the masses in kg, r is the distance between them in meters, and G is the universal gravitational constant: 6.67430×10^{-11} .

What is the direction? For the two moons, the force on moon 1 will pull toward moon 2. Likewise, the force on moon 2 will pull toward moon 1.

Of course, if something is big (like the sun), you need to be more specific: The force points directly at the center of mass of the object that is generating the force.

Each of the moons will start off with a velocity vector. That velocity vector will change over time as the moon is accelerated by the force of gravity. If you have a mass m with an initial velocity vector of $\vec{v}_0$ that is being accelerated with a constant force vector $\vec{F}$, at time t , the new velocity vector will be:

$$\vec{v}_t = \vec{v}_0 + \frac{t}{m} \vec{F}$$

If an object is at an initial position vector of $\vec{p}_0$ and moves with a constant velocity vector $\vec{v}$ for time t , the new position will be given by

$$\vec{p}_t = \vec{p}_0 + t\vec{v}$$

1.2 Simulations and Step Size

As two moons orbit each other, the force, acceleration, velocity, and position are changing smoothly and continuously. It is difficult to simulate truly continuous things on a digital computer.

However, think about how a movie shows you many frames each second. Each frame is a still picture of the state of the system. The more frames per second, the smoother it looks.

We do a similar trick in simulations. We say “We are going to run our simulation in 2 hour steps. We will assume that the acceleration and velocity were constant for those two hours. We will update our position vectors accordingly, then we will recalculate our acceleration and velocity vectors.”

Generally, as you make the step size smaller, your simulation will get more accurate and take longer to execute.

1.3 Make a Text-based Simulation

To start, you are going to write a Python program that simulates the moons and prints out their position for every time step. Later, we will add graphs and even animation.

We are going to assume the two moons are traveling the same plane so we can do all the math and graphing in 2 dimensions.

Each moon will be represented by a dictionary containing the state of the moon:

- Its mass in kilograms
- Its position — A 2-dimensional vector representing x and y coordinates of the center of the moon.
- Its velocity — A 2-dimensional vector
- Its radius — Each moon has a radius so we know when the centers of the two moons are so close to each other that they must have collided.
- Its color — We will use that when we do the plots and animations. One moon will be red, the other blue.

There will then be a loop where we will update the positions of the moons and then

recalculate the acceleration and velocities.

How much time will be simulated? 100 days or until the moons collide, whichever comes first.

We will use numpy arrays to represent our vectors.

Create a file called `moons.py`, and type in this code:

```
1 import numpy as np
2
3 # Constants
4 G = 6.67430e-11          # Gravitational constant (Nm2/kg2)
5 SEC_PER_DAY = 24 * 60 * 60 # How many seconds in a day?
6 MAX_TIME = 100 * SEC_PER_DAY # 100 days
7 TIME_STEP = 2 * 60 * 60   # Update every two hours
8
9 # Create the initial state of Moon 1
10 m1 = {
11     "mass": 6.0e22, # kg
12     "position": np.array([0.0, 200_000_000]), # m
13     "velocity": np.array([100.0, 25.0]), # m/s
14     "radius": 1_500_000.0, # m
15     "color": "red" # For plotting
16 }
17
18 # Create the initial state of Moon 2
19 m2 = {
20     "mass": 11.0e22, # kg
21     "position": np.array([0.0, -150_000_000]), # m
22     "velocity": np.array([-45.0, 2.0]), # m/s
23     "radius": 2_000_000.0, # m
24     "color": "blue" # For plotting
25 }
26
27 # Lists to hold positions and time
28 position1_log = []
29 position2_log = []
30 time_log = []
31
32 # Start at time zero seconds
33 current_time = 0.0
34
35 # Loop until current time exceed Max Time
36 while current_time <= MAX_TIME:
37
38     # Add time and positions to log
39     time_log.append(current_time)
40     position1_log.append(m1["position"])
41     position2_log.append(m2["position"])
42
```

```

43     # Print the current time and positions
44     print(f"Day {current_time/SEC_PER_DAY:.2f}:")
45     print(f"\tMoon 1:({m1['position'][0]:.1f},{m1['position'][1]:.1f})")
46     print(f"\tMoon 2:({m2['position'][0]:.1f},{m2['position'][1]:.1f})")
47
48     # Update the positions based on the current velocities
49     m1["position"] = m1["position"] + m1["velocity"] * TIME_STEP
50     m2["position"] = m2["position"] + m2["velocity"] * TIME_STEP
51
52     # Find the vector from moon1 to moon2
53     delta = m2["position"] - m1["position"]
54
55     # What is the distance between the moons?
56     distance = np.linalg.norm(delta)
57
58     # Have the moons collided?
59     if distance < m1["radius"] + m2["radius"]:
60         print(f"*** Collided {current_time:.1f} seconds in!")
61         break
62
63     # What is a unit vector that points from moon1 toward moon2?
64     direction = delta / distance
65
66     # Calculate the magnitude of the gravitational attraction
67     magnitude = G * m1["mass"] * m2["mass"] / (distance**2)
68
69     # Acceleration vector of moon1 (a = f/m)
70     acceleration1 = direction * magnitude / m1["mass"]
71
72     # Acceleration vector of moon2
73     acceleration2 = (-1 * direction) * magnitude / m2["mass"]
74
75     # Update the velocity vectors
76     m1["velocity"] = m1["velocity"] + acceleration1 * TIME_STEP
77     m2["velocity"] = m2["velocity"] + acceleration2 * TIME_STEP
78
79     # Update the clock
80     current_time += TIME_STEP
81
82     print(f"Generated {len(position1_log)} data points.")

```

When you run the simulation, you will see the positions of the moons for 100 days:

```

1  > python3 moons.py
2  Day 0.00:
3      Moon 1:(0.0,200,000,000.0)
4      Moon 2:(0.0,-150,000,000.0)
5  Day 0.08:
6      Moon 1:(720,000.0,200,180,000.0)
7      Moon 2:(-324,000.0,-149,985,600.0)
8  Day 0.17:

```

```
9         Moon 1:(1,439,990.7,200,356,896.1)
10        Moon 2:(-647,995.0,-149,969,507.0)
11    ...
12    Day 100.00:
13        Moon 1:(119,312,305.5,283,265,313.5)
14        Moon 2:(17,393,287.9,-60,319,261.9)
15    Generated 1201 data points.
```

Look over the code. Make sure you understand what every line does.

1.4 Graph the Paths of the Moons

Now, you will use the matplotlib to graph the paths of the moons. Add this line to the beginning of `moons.py`.

```
1 import matplotlib.pyplot as plt
```

Add this code to the end of your `moons.py`:

```
1  # Convert lists to np.arrays
2  positions1 = np.array(position1_log)
3  positions2 = np.array(position2_log)
4
5  # Create a figure with a set of axes
6  fig, ax = plt.subplots(1, figsize=(7.2, 10))
7
8  # Label the axes
9  ax.set_xlabel("x (m)")
10 ax.set_ylabel("y (m)")
11 ax.set_aspect("equal", adjustable='box')
12
13 # Draw the path of the two moons
14 ax.plot(positions1[:, 0], positions1[:, 1], m1["color"], lw=0.7)
15 ax.plot(positions2[:, 0], positions2[:, 1], m2["color"], lw=0.7)
16
17 # Save out the figure
18 fig.savefig("plotmoons.png")
```

When you run it, your `plotmoons.png` should look like this:

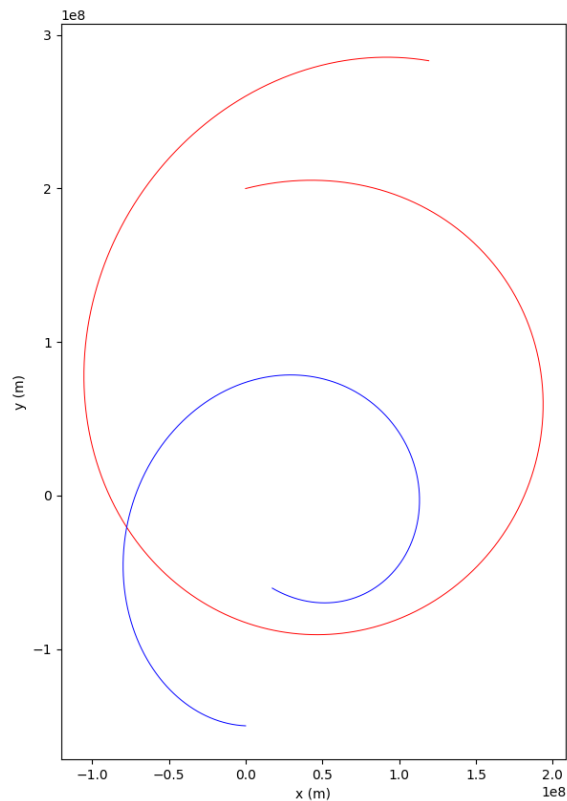


Figure 1.1: Output of moons_01.py.

It is nifty to see the paths, but we don't know where each moon was at a particular time. In fact, it is difficult to figure out which end of each curve was the beginning and which was the ending.

What if we added some lines and labels every 300 steps to put a sense of time into the plot? Add one more constant after the import statements:

```
1 PAIR_LINE_STEP = 300 # How time steps between pair lines
```

Immediately before you save the figure to the file, add the following code:

```
1 # Draw some pair lines that help the
2 # viewer understand time in the graph
3 i = 0
4 while i < len(positions1):
5
6     # Where are the moons at the ith entry?
7     a = positions1[i, :]
8     b = positions2[i, :]
9     ax.plot([a[0], b[0]], [a[1], b[1]], "--", c="gray", lw=0.6, marker=".")
```

```

10
11     # What is the time at the ith entry?
12     t = time_log[i]
13
14     # Label the location of moon 1 with the day
15     ax.text(a[0], a[1], f"{t/SEC_PER_DAY:.0f} days")
16     i += PAIR_LINE_STEP

```

When you run it, your plot should look like this:

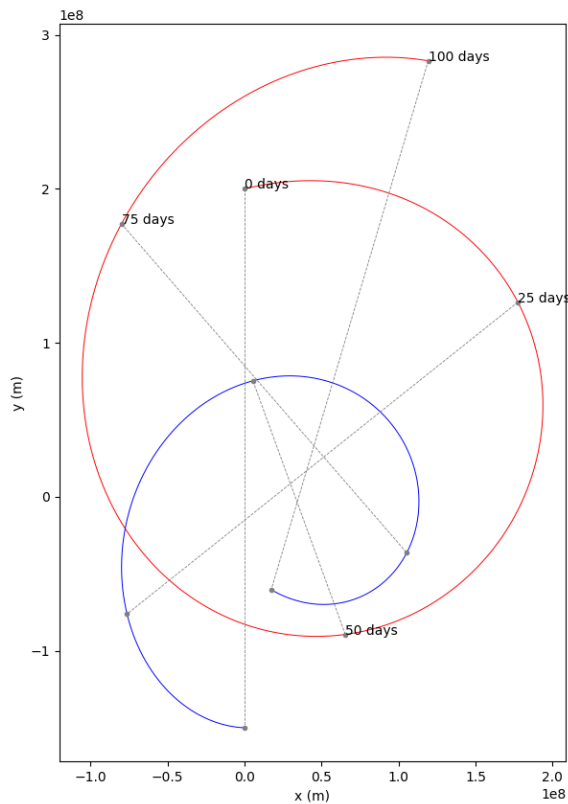


Figure 1.2: Plot moons_02.py.

Now you can get a feel for what happened. The moons were attracted to each other by gravity and started to circle each other. The heavier moon accelerates less quickly, so it makes a smaller loop.

Maybe we will get a better feel for what is happening if we look at more time. Let's increase it to 400 days. Change the relevant constant:

```

1 MAX_TIME = 400 * SEC_PER_DAY # 100 days

```

Now it should look like this:

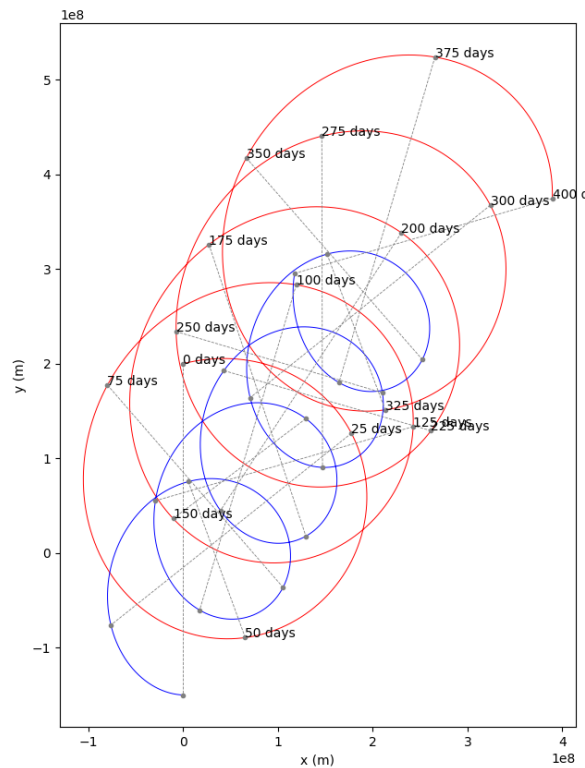


Figure 1.3: Output of plot moons_03.py.

Now you can see the pattern. They are rotating around each other and the pair is gradually migrating up and to the right.

1.5 Conservation of Momentum

Recall the idea of momentum being conserved in a system. In these programs, you are observing an extra important idea: *the momentum of a system will be conserved*. That is, absent forces from outside the system, the velocity of the center of mass will not change.

We can compute the initial center of mass and its velocity. In both cases, we just do a weighted average using the mass of the moon as the weight.

Immediately after you initialize the state of two moons, calculate the initial center of mass and its velocity:

```

1  # Calculate the initial position and velocity of the center of mass
2  tm = m1["mass"] + m2["mass"] # Total mass
3  cm_position = (m1["mass"] * m1["position"] + m2["mass"] * m2["position"]) / tm

```

```
4 cm_velocity = (m1["mass"] * m1["velocity"] + m2["mass"] * m2["velocity"]) / tm
```

Let's record the center of mass for each time. Before the loop starts, create a list to hold them:

```
1 cm_log = []
```

Inside the loop (before any calculations), append the current center of mass position to the log:

```
1 cm_log.append(cm_position)
```

Anywhere later in the loop (after you update the positions of the moon), update `cm_position`:

```
1 # Update the center of mass
2 cm_position = cm_position + cm_velocity * TIME_STEP
```

Now, let's look at the positions of the moons relative to the center of mass. Before you do any plotting, convert the list to a numpy array and subtract it from the positions:

```
1 cms = np.array(cm_log)
2
3 # Make positions relative to the center of mass
4 positions1 = positions1 - cms
5 positions2 = positions2 - cms
```

When you run it, you can really see what is happening:

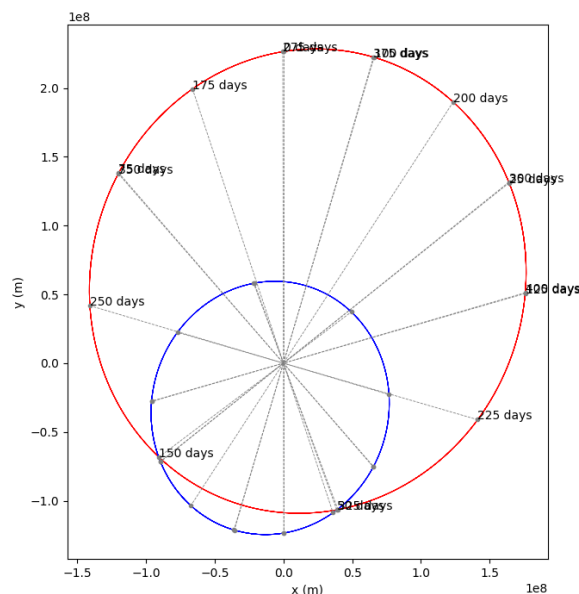


Figure 1.4: Output of plotmoons_04.py.

The moons are tracing elliptical paths. The center of mass is the focus point for both of them.

1.6 Animation

One of the features of matplotlib that not a lot of people understand is how to make animations with it. This seems like a really great opportunity to make an animation showing the position, velocity, acceleration of the moons. We will also show the center of mass.

The trick to animations is that you create a bunch “artist” objects. You create a function that updates the artists. matplotlib will call your functions, tell the artists to draw themselves, and make a movie out of that.

Make a copy of moons.py called animate_moons.py.

Edit it to look like this:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Import animation support and artists
5 from matplotlib.animation import FuncAnimation
6 from matplotlib.patches import Circle, FancyArrow

```

```

7  from matplotlib.text import Text
8
9  # Constants
10 G = 6.67430e-11 # Gravitational constant (Nm2/kg2)
11 SEC_PER_DAY = 24 * 60 * 60 # How many seconds in a day?
12 MAX_TIME = 400 * SEC_PER_DAY # 100 days
13 TIME_STEP = 12 * 60 * 60 # Update every 12 hours
14 FRAMECOUNT = MAX_TIME / TIME_STEP # How many frames in animation
15 ANI_INTERVAL = 1000 / 50 # ms for each frame in animation
16
17 # The velocity and acceleration vectors are invisible
18 # unless we scale them up. A lot.
19 VSCALE = 140000.0
20 ASCALE = VSCALE * 800000.0
21
22 # Create the initial state of Moon 1
23 m1 = {
24     "mass": 6.0e22, # kg
25     "position": np.array([0.0, 200_000_000]), # m
26     "velocity": np.array([100.0, 25.0]), # m/s
27     "radius": 1_500_000.0, # m
28     "color": "red", # For plotting
29 }
30
31 # Create the initial state of Moon 2
32 m2 = {
33     "mass": 11.0e22, # kg
34     "position": np.array([0.0, -150_000_000]), # m
35     "velocity": np.array([-45.0, 2.0]), # m/s
36     "radius": 2_000_000.0, # m
37     "color": "blue", # For plotting
38 }
39
40 # Calculate the initial position and velocity of the center of mass
41 tm = m1["mass"] + m2["mass"] # Total mass
42 cm_position = (m1["mass"] * m1["position"] + m2["mass"] * m2["position"]) / tm
43 cm_velocity = (m1["mass"] * m1["velocity"] + m2["mass"] * m2["velocity"]) / tm
44
45 # Start at time zero seconds
46 current_time = 0.0
47
48 # Create the figure and axis
49 fig, ax = plt.subplots(1, figsize=(7.2, 10))
50
51 # Set up the axes
52 ax.set_xlabel("x (m)")
53 ax.set_xlim((-1.2e8, 4e8))
54 ax.set_ylabel("y (m)")
55 ax.set_ylim((-1.6e8, 5.5e8))
56 ax.set_aspect("equal", adjustable="box")
57 fig.tight_layout()
58
59 # Create artists that will be edited in animation

```

```

60 time_text = ax.add_artist(Text(0.03, 0.95, "", transform=ax.transAxes))
61 circle1 = ax.add_artist(Circle((0, 0), radius=m1["radius"], color=m1["color"]))
62 circle2 = ax.add_artist(Circle((0, 0), radius=m2["radius"], color=m2["color"]))
63 circle_cm = ax.add_artist(Circle((0, 0), radius=m2["radius"], color="purple"))
64 varrow1 = ax.add_artist(FancyArrow(0, 0, 0, 0, color="green",
↳ head_width=m1["radius"]))
65 varrow2 = ax.add_artist(FancyArrow(0, 0, 0, 0, color="green",
↳ head_width=m2["radius"]))
66 acc_arrow1 = ax.add_artist(
67     FancyArrow(0, 0, 0, 0, color="purple", head_width=m1["radius"])
68 )
69 acc_arrow2 = ax.add_artist(
70     FancyArrow(0, 0, 0, 0, color="purple", head_width=m2["radius"])
71 )
72
73
74 # This function will get called for every frame
75 def animate(frame):
76
77     # Global variables needed in scope from the model
78     global cm_position, cm_velocity, current_time, m1, m2
79
80     # Global variables needed in scope from the artists
81     global time_text, varrow1, varrow2, acc_arrow1, acc_arrow2, circle1, circle2,
↳ circle_cm
82
83     print(f"Updating artists for day {current_time/SEC_PER_DAY:.1f}.")
84
85     # Update the positions based on the current velocities
86     m1["position"] = m1["position"] + m1["velocity"] * TIME_STEP
87     m2["position"] = m2["position"] + m2["velocity"] * TIME_STEP
88
89     # Update day label
90     time_text.set_text(f"Day {current_time/SEC_PER_DAY:.0f}")
91
92     # Update positions of circles
93     circle1.set_center(m1["position"])
94     circle2.set_center(m2["position"])
95
96     # Update velocity arrows
97     varrow1.set_data(
98         x=m1["position"][0],
99         y=m1["position"][1],
100         dx=VSCALE * m1["velocity"][0],
101         dy=VSCALE * m1["velocity"][1],
102     )
103     varrow2.set_data(
104         x=m2["position"][0],
105         y=m2["position"][1],
106         dx=VSCALE * m2["velocity"][0],
107         dy=VSCALE * m2["velocity"][1],
108     )
109

```

```

110     # Update the center of mass
111     cm_position = cm_position + cm_velocity * TIME_STEP
112     circle_cm.set_center(cm_position)
113
114     # Find the vector from moon1 to moon2
115     delta = m2["position"] - m1["position"]
116
117     # What is the distance between the moons?
118     distance = np.linalg.norm(delta)
119
120     # Have the moons collided?
121     if distance < m1["radius"] + m2["radius"]:
122         print(f"*** Collided {current_time:.1f} seconds in!")
123
124     # What is a unit vector that points from moon1 toward moon2?
125     direction = delta / distance
126
127     # Calculate the magnitude of the gravitational attraction
128     magnitude = G * m1["mass"] * m2["mass"] / (distance**2)
129
130     # Acceleration vector of moons (a = f/m)
131     acceleration1 = direction * magnitude / m1["mass"]
132     acceleration2 = (-1 * direction) * magnitude / m2["mass"]
133
134     # Update the acceleration arrows
135     acc_arrow1.set_data(
136         x=m1["position"][0],
137         y=m1["position"][1],
138         dx=ASCALE * acceleration1[0],
139         dy=ASCALE * acceleration1[1],
140     )
141     acc_arrow2.set_data(
142         x=m2["position"][0],
143         y=m2["position"][1],
144         dx=ASCALE * acceleration2[0],
145         dy=ASCALE * acceleration2[1],
146     )
147
148     # Update the velocity vectors
149     m1["velocity"] = m1["velocity"] + acceleration1 * TIME_STEP
150     m2["velocity"] = m2["velocity"] + acceleration2 * TIME_STEP
151
152     # Update the clock
153     current_time += TIME_STEP
154
155     # Return the artists that need to be redrawn
156     return (
157         time_text,
158         varrow1,
159         varrow2,
160         acc_arrow1,
161         acc_arrow2,
162         circle1,

```

```

163         circle2,
164         circle_cm,
165     )
166
167
168     # Make the rendering happen
169     animation = FuncAnimation(
170         fig,
171         animate,
172         np.arange(FRAMECOUNT),
173         interval=ANI_INTERVAL
174     )
175
176     # Save the rendering to a video file
177     animation.save("moonmovie.mp4")

```

When you run this, it will take longer than the previous versions. You should have a video file that shows a simulation of the moons tracing their elliptical paths around their center of mass:

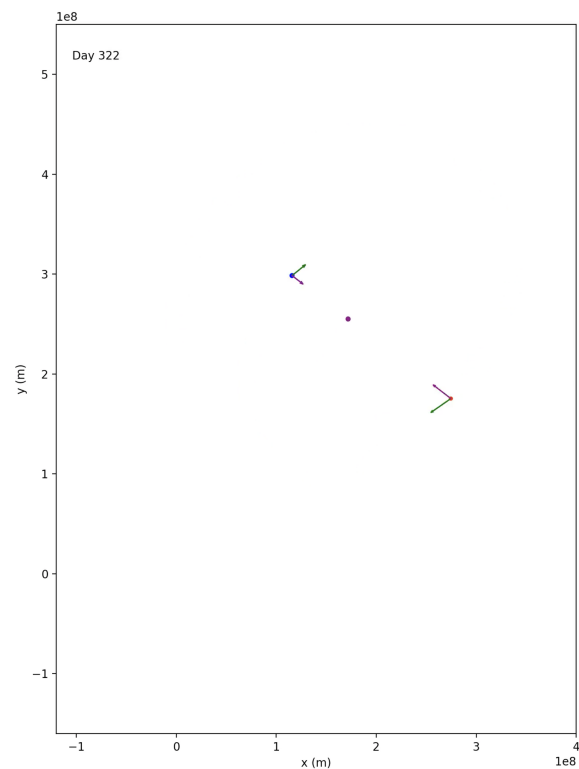


Figure 1.5: A still of the animation produced.

1.7 Challenge: The Three-Body Problem

It is time to stretch a little as a physicist and programmer: You are going to make a new version of `moons.py` that handles three moons instead of just two.

This is known as “The Three-Body Problem”, and people have tried for centuries to come up with a way to figure out (from the initial conditions) where the three moons would be at time t without doing a simulation. And no one has.

For a lot of problems, the outcome is not very sensitive to the initial conditions. For example, consider the flight of a cannonball: If it leaves the muzzle of the cannon a little faster, it will go a little farther.

For the three-body problem, the outcome can be radically different even if the initial conditions are very similar.

(There is a whole field of mathematics studying systems that are very sensitive to initial conditions. It is known as *dynamical systems* or *chaos theory*.)

Copy `moons.py` to `3moons.py`. Here is a reasonable initial state for your third moon:

```
1 m3 = {  
2     "mass": 4.0e22, # kg  
3     "position": np.array([50_000_000, 80_000_000]), # m  
4     "velocity": np.array([-30.0, -35.0]), # m/s  
5     "radius": 1_700_000.0, # m  
6     "color": "green"  
7 }
```

If we run that simulation for 100 days, we get a plot like this:

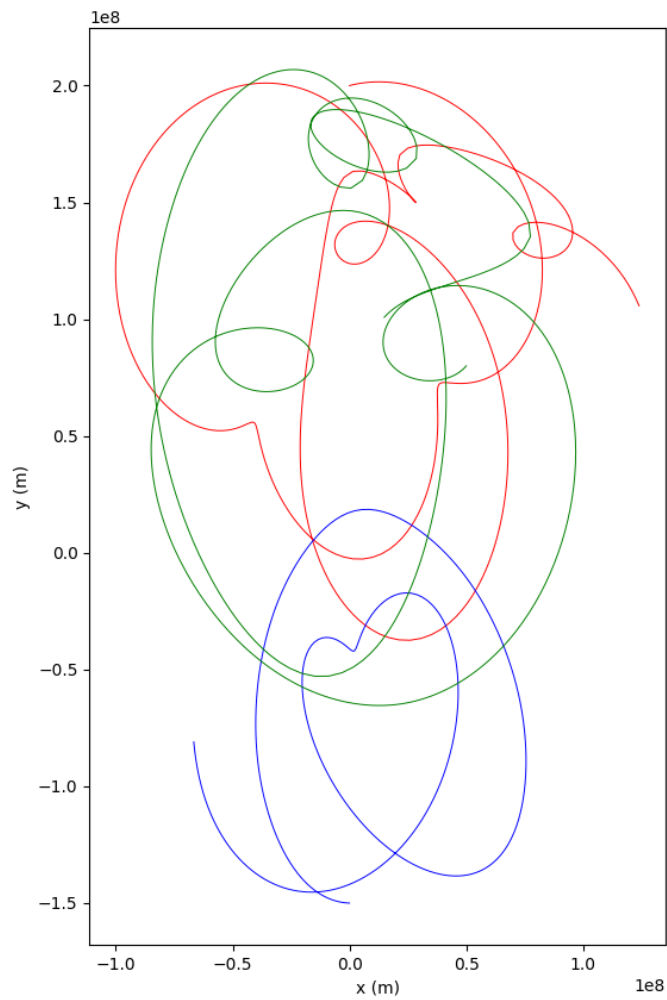


Figure 1.6: The Three Body Problem.

Visibly, you can see this is very different from the two-body problem that just traced ellipses around the center of mass.

This is a draft chapter from the Kontinua Project. Please see our website (<https://kontinua.org/>) for more details.

Answers to Exercises



INDEX

chaos theory, [15](#)

dynamical systems, [15](#)

gravitation, [1](#)

Three-Body Problem, [15](#)